

# JEDEC standards

Pinouts:

CK-t, CK-c input differential clock

CKE input synchronous  
does not matter

CS-n input ↓ chip select (rank select)

C0, C2, C2 input ↓ not important  
Chip ID (

ODT input On-die-termination

ACT-n input Activation command  
input into RAS-n/A16,  
CAS-n/A15,  
WE-n/A14

RAS-n/A16, CAS-n/A15  
WE-n/A14 Input define command being  
entered can define  
row address when ACT-n  
low.

DM-h/DBI-h/

TDOs-t, (DMU-h/ I/O

DBIU-h), (DML-h/

DBI-h)

Input data  
marked and data  
bus inversion: DM-h  
is a master for write  
data. Marked when  
DM-h low.

DBI-h determines  
whether data is  
inverted (low) or not (high).

TDOs is for sampling  
DM-h which is sampled  
on both edges.

B60-B62

Input

Determines which bank  
group to select for command.

BAU-BA1

Input

determines which bank  
to select for command.

A10/AP

Input

auto-percentage

A12/BC-h

Input

burst chop (active low)

RESET-h

Input

what do you think?

DQ  $\Sigma/\text{O}$

data bus

DQS<sub>-t</sub>,  
DQS<sub>-c</sub>, DQS<sub>U-c</sub>  
DQS<sub>L-t</sub>, DQS<sub>L-c</sub>

$\Sigma/\text{O}$

Differential data  
strobe signals, Edge Aligned  
for reads, centered for  
writes.

TDQS<sub>-t</sub>, TDQS<sub>-c</sub>

output

termination data strobe

PAR

Input

Even parity

ALERT<sub>-n</sub>

$\Sigma/\text{O}$

CRC error

TEN

Input

connectivity test



## State Machine Summary

Idle - to Activating

to bank active, to reading, to writing, to precharge, back to idle.

Read and Write can loop back on themselves or each other for row hits, otherwise they must precharge and return to idle.

## Mode registers

Written into registers using similar transactions as read and write. They have to be in modes not allowing for reading or writing.

Can configure read burst type, burst length, CAS latency, etc.

Can also configure write recovery and read to precharge when auto-precharge enabled.

## Mode Register 1

- Can configure ~~MR~~, T1DAS, DCL, Write leveling enable

## Mode Register 2

- Write CRC, CAS write latency, low power any self refresh,

## Mode Register 3

- MPR read format

- temp sensor

- Write DRD latency

## MR 4

- self refresh abort

- temperature control refresh mode

## MR 5

- Read/Write OBI

- Data mask

- CR parity

- CR err clear

- rC/b parity latency mode

M.R. 6

ZBK

When you want to set a mode register, all CAS, RAS, WE are set high while appropriate pins are set to the opcodes to be fed into the Mode Registers.

+CK.SRE time self refresh

A DLL is used to keep synchronized with input clock.

010 change clock frequency, turn off

DLL, enter self refresh, change

input freq. wait and have

stable clock for +CKSRX, exit

SR, and update mode registers

accordingly. TCLK needs programmed

DLL is then reset



Write leveling is using the DQS signals of the memory controller to align DQS signals with clock.

- Controller enables write leveling mode. Mode register command
- DRAM makes sure DQS edge is aligned with clock, it sends back DQ data (not stored) during write leveling mode to ODR helps controller account for additional skew.
- Controller and DRAM device now know the correct alignment and delay for write transfers on DQS.
- Once correct, controller locks DQ L+DQS-c delay setting.
- DQ is now undefined, until an MR2 command, which after +MOD, we'll enable DQ to be used again.

MPR (multipurpose register) in MR3 can be written to and read from for storing data I think?

- Address using B pins and written to using A pins as data.

Small snippet

Data Mask DBI and TDOs  
Data mask ~~DBI~~ and DBI cannot be enabled simultaneously. If both are disabled, DRAM turns off its ~~receiver~~ receiver. If DBI is disabled, DRAM turns off its transmitter.

CAL Mode (S<sub>0</sub> to Command Address Latency)

Configurable latency, usually between 3-8 clock cycles. When enabled and writing to MR, needs to wait  $t_{MOD-CAL} = t_{MOD} + t_{CAL}$  until next transaction.

DDR 4 CRC polynomial  $x^8 + x^2 + x + 1$

0 For 72 bits of data. Combinational logic implementation uses 272 two-input XOR gates and eight 6 gate deep trees.



from  
memory  
low  
memory

1 If DRAM detects a CRC error,  
it will send +CRC-ALERT ~~after~~ after  
that time. It will also write a one  
CRC error clear bit in MRS, which  
remains that way until host explicitly  
clears it using MRS command.  
Controller is responsible for fixing  
error in coherent manner.

2 After parity error, DRAM sets  
**ALERT-n** after +PAR-ALERT  
and keeps it on for +PAR-ALERT-OW +  
+PAR-ALERT-PW. DRAM cannot execute  
anything or set the corresponding register  
low until after the above time.  
Parity error status is still one, but  
can be reset and taken care of  
by controller, has to be reset to  
enable parity checking again.